Week 3 - Revision

Verilog fundamentals, simulation, coverage and the Bambu HLS tutorial

Course Lessons 11-14: Verilog I and II, functional verification using simulation, and Tutorial 3. Ten typed revision pages are followed by six original handwritten pages. Read widths and event ordering before predicting an answer; assignment choices are not marked here. [1-5]

1. Types | 2. Widths | 3. Operators | 4. Modules | 5. Time | 6. Assignments | 7. Queues | 8. Coverage | 9. HLS | 10. Recall/sources | Your handwriting

1. HDL thinking, declarations and four-state values

An **HDL** describes concurrent hardware, connections, state and timing. A simulator evaluates the model; synthesis translates a supported subset into hardware. Successful simulation does not make every construct synthesizable. A testbench delay does not specify a manufactured gate delay. Different processes are concurrent, while statements inside one `begin ... end` execute in their defined order. [2]

Declaration or value	Meaning and common misunderstanding
`wire`	A net derives its value from its drivers; an ordinary undriven wire reads `z`
`reg`	A procedural variable retains its last assigned simulation value; uninitialized it reads `x`
`wire [7:0] bus`	One eight-bit vector; `bus[3:0]` selects four bits
`reg [7:0] mem [0:15]`	Sixteen words of eight bits each; 128 data bits in total
`integer`, `time`, `real`	Traditional signed 32-bit integer, unsigned 64-bit time value, and floating-point simulation value
`0`, `1`, `x`, `z`	Known low, known high, unknown, and high impedance; these are simulation states

`reg` **does not mean flip-flop.** A complete combinational process can assign a `reg` without storing state. An edge-triggered process can describe flip-flops; an incomplete combinational assignment can require retention and infer a latch. The process and assignment paths determine the behaviour.

Unknown versus released: `x` indicates insufficiently known information, such as uninitialized state or conflicting equal-strength drivers. `z` describes a released drive contribution. A driven `1` and a released `z` can resolve to `1`; `z` is not a second spelling of `x`. For operator evaluation, controlling bits can give known results: `0 & x` is 0 and `1 | x` is 1.

Verilog is case-sensitive. Keywords are lowercase; ordinary identifiers begin with a letter or underscore. Use `//` for a line comment and `/* ... */` for a block comment. `[7:0]` and `[0:7]` are both legal ranges, but their index directions differ. Do not infer bit significance from an index number alone.

2. Literals, width, truncation and signedness

A based integer literal is **size, apostrophe, optional `s`, base, digits**, for example `8'shF6`. Size counts bits: binary, octal and hexadecimal digits do not change the declared width. Underscores are readability separators. A leading minus is a unary operator, not a digit inside the literal. [2]

Example	Stored bits or result	Why
`8'h3C`	`00111100`, unsigned 60	Each hexadecimal digit supplies four bits
`6'hA5`	`100101`, unsigned 37	Six bits retain the least significant six of `10100101`
`8'b101`	`00000101`	Ordinary positive digits are zero-padded on the left
`8'bz01`	`zzzzzz01`	A leading `z` pads the unfilled upper positions with `z`
`8'bx01`	`xxxxxx01`	A leading `x` similarly extends unknown positions
`8'shF6`	`11110110`, signed -10	Signed interpretation uses the top bit and two's complement
Same bits treated unsigned	246	The bits alone do not determine signed interpretation

Two's complement: at eight bits, -10 has the encoding $256 - 10 = 246$. This explains the bit pattern without confusing signed value and unsigned encoding. Narrowing discards upper bits; it is not rounding or saturation.

Preserve carry explicitly. For eight-bit unsigned inputs, the largest sum is $255 + 255 = 510$, requiring nine result bits. Write `sum = {1'b0,a} + {1'b0,b};` with a nine-bit destination. At `a=240`, `b=32`, the full sum is 272; the low eight bits alone hold 16. A wider destination cannot recover information already lost in a self-determined narrow intermediate.

Signed operations need a signed context. For signed eight-bit `s=-10`, an explicitly signed temporary assigned `s >>> 1` holds -5. `>>` shifts in zeros; `>>>` sign-extends only when the effective left operand is signed. Mixed signed/unsigned expressions can change extension and interpretation. Use deliberate signed declarations/constants and inspect intermediate widths.

Unsized decimal literals are signed and at least 32 bits in traditional Verilog. String literals use eight bits per character: five characters need 40 bits in a traditional packed-vector representation. Type, width and signedness are separate questions.

3. Operators, equality, unknowns and wildcard cases

First identify whether an operator works on individual bits, reduces one vector, or tests truth. Assume the explicitly sized binary values below. [3]

Expression	Result	Interpretation
`4'b0110 & 4'b1010`	`4'b0010`	Bitwise AND at each position
`4'b0110 && 4'b1010`	`1'b1`	Logical AND: both operands are nonzero
`&4'b1110`	`1'b0`	Reduction AND produces one bit
`^4'b1100`	`1'b0`	Reduction XOR: an even number of ones
`~4'b0110` / `!4'b0110`	`4'b1001` / `1'b0`	Bitwise complement / logical negation
`{3'b101,2'b10}`	`5'b10110`	Left item occupies the more significant portion
`{3{2'b01}}`	`6'b010101`	Replicate the two-bit pattern three times
`1'bx == 1'bx` / `1'bx === 1'bx`	`x` / `1`	Indeterminate logical equality / exact case equality

Conditional operator: with a one-bit unknown select, equal-width integral alternatives merge bit by bit. `1'bx ? 4'b1100 : 4'b1010` gives `4'b1xx0`. Equal bits survive; disagreements become unknown. A procedural `if` with an unknown one-bit condition does not take its true branch. That can hide an unknown control instead of exposing the ambiguity.

Construct	Matching rule
`case`	Exact four-state matching
`casez`	`z`/`?` positions are ignored in the selector or item
`casex`	`x`, `z` and `?` positions are ignored in the selector or item

In a based literal, `?` encodes `z`; it is not a fifth logic state or a universal wildcard. Wildcard behaviour comes from the surrounding case construct. An unintended `z` selector can broaden `casez` matches; `casex` can conceal uninitialized `x` control bits. Define intentional patterns and defaults.

Exam method: record each operand's width and signedness, apply the operator's four-state rule, then handle destination conversion. Parentheses make precedence clear but do not automatically widen operands. Unknown simulation values are not freely chosen synthesis don't-cares.

4. Modules, parameters, functions and tasks

A **module definition** is a type; an **instance** is one use with its own connections and state. `input`, `output` and `inout` describe port direction. Named connections `.a(parent_a)` connect the child's port `a` to the parent's signal `parent_a`; the two names need not match. [3]

A **parameter** is selected during elaboration. It can configure widths or structure and can be overridden per instance. It is not a changing runtime signal. `localparam` defines an internal constant that an instance cannot override. A generate condition selects structure at elaboration; a procedural `if` can describe behaviour during execution.

```
Verilog
module pass #(parameter W=4)
  (input wire [W-1:0] a, output wire [W-1:0] y);
  assign y = a;
endmodule

module top(input wire [7:0] data, output wire [7:0] out);
  pass #(.W(8)) u_pass (.a(data), .y(out));
endmodule
```

The instance has eight-bit ports because `W=8` is resolved before execution. Changing `data` does not change `W`. A continuous `assign` at module scope drives a net whenever its expression changes. It is not an ordinary statement inserted into an `always` block. Legacy procedural `assign/deassign` are a separate language facility.

Traditional Verilog rule	Function	Task
Result and use	One return value; usable in an expression	Called as a statement; output/inout arguments can return values
Arguments	At least one input; input arguments only	Zero or more input/output/inout arguments
Timing controls	Cannot suspend with `#`, `@` or `wait`	May contain timing/event controls
Calls	Other functions, not tasks	Functions or tasks

A task is permitted to take simulation time; it need not do so. A function's zero simulation time does not mean zero hardware delay or zero computer effort. Neither label alone determines synthesizability or circuit storage. SystemVerilog extends these traditional rules. Default local storage is static; `automatic` gives separate invocation storage where needed.

5. Processes, sensitivity lists, edges and time

An `initial` **block** starts once and finishes when its statements finish. It can contain waits/delays, so it need not finish at time zero. An `always` **block** repeats its body. Separate processes start concurrently; source order does not give one priority. `begin/end` sequences statements within a process; `fork/join` runs concurrent branches and waits for all branches. [3]

Sensitivity is an event condition. `@(a or b)` resumes on a change of either signal; the `or` separates events, not arbitrary gate operators. `always @(enable)` does not continually run while enable is high and does not react to every other value read in its body. Use `always @*` for ordinary combinational processes, with every output assigned on every path.

Single-bit event	Transitions that count
`posedge`	`0->1`, `0->x`, `0->z`, `x->1`, `z->1`
`negedge`	`1->0`, `1->x`, `1->z`, `x->0`, `z->0`
Neither	`x->z`, `z->x`, or no change

These are simulation rules, not a promise that an uncertain electrical clock is safe. In an edge expression that is a vector, edge detection uses its least significant bit; normal clocks should be one bit.

```Verilog
`timescale 1ns/1ps
module clock_demo;
  reg clk = 1'b0;
  always #4 clk = ~clk;
  initial #25 $finish;
endmodule
```

After initialization, clock toggles occur at 4, 8, 12, 16, 20 and 24 ns. Four nanoseconds is the half-period; period is 8 ns and frequency is 125 MHz. An uninitialized `x` clock remains `x` when complemented. A repeating process without a delay/event control can prevent time from advancing.

Time unit versus precision: `1ns/1ps` means delay values use nanoseconds and are rounded to 1 ps resolution. `#2.3456` therefore requests 2.3456 ns and rounds to 2.346 ns. Simulation time is modeled circuit time; wall-clock runtime is how long the computer takes. A `for` loop uses a clock cycle per iteration only if its body explicitly waits for those events.

6. Blocking, nonblocking and delayed assignments

Blocking = completes the assignment before the next statement in that process. **Nonblocking** <= evaluates the RHS when reached and schedules a later LHS update; without an explicit delay, that update belongs to the NBA region of the current time slot. It does not automatically wait for the next clock or the end of the entire simulation. [3,4]

For a two-stage clocked pipeline, assume known reset values, inputs stable before each rising edge, and observations after NBA updates:

```verilog
always @(posedge clk) begin
  q1 <= d;
  q2 <= q1;
end
```

Edge	Input `d`	Old `q1`	New `q1`	New `q2`
First, after reset	12	0	12	0
Second	7	12	7	12
Third	31	7	31	7

Both RHS values are sampled before these pending updates. Replacing both assignments with = in the same order lets `q2` see the newly assigned `q1`, changing the behaviour. Separate blocking clocked processes can also race. Use blocking assignments for ordinary combinational work and nonblocking assignments for clocked state, while checking the whole process.

Delayed assignments: execute each sequence from time zero, with constant RHS values and a 1 ns time unit:

Sequence inside one process	Times when the destinations become one
`a = #3 1; b = #8 1; c = #5 1;`	`a`: 3 ns; `b`: 11 ns; `c`: 16 ns
`p <= #3 1; q <= #8 1; r <= #5 1;`	`p`: 3 ns; `q`: 8 ns; `r`: 5 ns

The blocking waits accumulate. The nonblocking writes are all scheduled while the process runs at time zero. With `q <= #8 d`, sample `d` now and delay the write. With `#8 q <= d`, wait first and sample `d` later. If `d` changes from 0 to 1 at 4 ns, the two forms write 0 and 1 respectively at 8 ns. These explicit delays are simulation models, not synthesizable arbitrary delay elements.

7. Simulation queues, observations and races

Event-driven simulation evaluates activated processes and applies updates, which can activate more processes. It settles eligible work at a timestamp before moving to future scheduled times. Several event iterations can occur without time advancing. The lecture's time-wheel example (`T % m`) illustrates an implementation data structure, not a mandatory simulator architecture. [4]

Classic Verilog region/category	Typical activity
Active	Ordinary process execution, blocking updates, continuous-assignment evaluation, NBA RHS evaluation
Inactive	Continuation after procedural `#0`
NBA update	Apply previously scheduled nonblocking LHS values
Monitor	Settled observations by `$strobe` and `$monitor`
Future events	Become eligible at later simulation times

An NBA update can awaken more active work at the same time. This is not a one-pass queue diagram. The table is the classic Verilog model used in the course; SystemVerilog has additional scheduling regions.

```
Verilog
`timescale 1ns/1ps
module queue_check;
  reg a;
  initial begin
    a = 0;
    a <= 1;
    $display("active a=%b", a);
    $strobe("settled a=%b", a);
    #1 $finish;
  end
endmodule
```

The display prints `active a=0`; the strobe prints `settled a=1`. Both belong to time zero. `$display` observes when executed; `$strobe` waits for settled end-of-slot observation. `$monitor` reports changes to its listed values. A procedural `#0` is still before NBA updates and is not a universal way to sample new clocked state.

A **race** makes the result depend on an unspecified ordering of eligible processes. Driving an input with = at the same edge that the DUT samples it can race. In a small testbench, drive on the opposite edge and check after the NBA updates; a suitable SystemVerilog clocking block is another approach. Races are unrelated to any fixed coverage percentage.

Cycle-based simulation evaluates synchronous next-state behaviour at clock boundaries and can reduce scheduling overhead for suitable models. Its abstractions must fit the design; it is not the default answer for arbitrary asynchronous or fine-delay behaviour. Neither simulation style proves an untested specification.

8. Testbenches, waveforms and coverage

The **DUT** is the design under test. The **testbench** generates stimulus, establishes clock/reset conditions and compares responses with an expected model. A **simulator** executes the design and testbench; a waveform viewer displays recorded activity. A plausible-looking trace is weaker evidence than a check of the actual requirement. [4]

Coverage or evidence	Question answered	Limitation
Statement/line coverage	Did the measured source statements execute?	Executed does not mean correct
Branch coverage	Were the control alternatives exercised?	Does not establish all data combinations
FSM state / transition coverage	Which states / which transitions were reached?	Visiting endpoint states does not prove a particular transition occurred
Toggle coverage	Did measured bits make the required transitions?	Does not show that every feature works
Functional coverage	Did specified scenarios/bins occur?	Requires an explicit, adequate coverage model
Self-checking result	Did an observed response match the expected result?	Establishes only the checks actually performed

Why 100% code coverage is insufficient: a missing feature can have no RTL line to mark uncovered. An adder can execute on every test while the checker ignores its carry output. Functional coverage is usually more manual because someone must specify meaningful scenarios and crosses. Coverage and correctness are complementary evidence.

For an enabled counter, check reset from a nonzero state, increment at each enabled edge, retention while disabled and wrap at the maximum value. A final-value-only comparison can miss a skipped increment followed by a double increment. Drive stable inputs before sampling and compare after the DUT's NBA updates have settled.

Unknown-sensitive checker: `if (actual !== expected)` treats unexpected `x` or `z` as a mismatch when the expectation is known. `!=` can itself evaluate to `x`, allowing an ordinary `if` to miss the intended failure branch.

VCD = Value Change Dump. `$dumpfile` names the output and `$dumpvars` selects recorded hierarchy. VCD stores times and values, not a pass/fail proof. `$finish` ends simulation; `$stop` requests a stop whose interactive behaviour depends on the simulator. Successful compilation/elaboration alone does not establish a passing simulation. [7]

9. Bambu: C behaviour to a timed RTL architecture

Tutorial 3 introduces **Bambu**, a high-level synthesis tool. It takes supported high-level behaviour and implementation constraints, selects resources, schedules and binds operations, and generates RTL control, datapath and interfaces. The algorithm does not uniquely specify the number of arithmetic units, cycle latency or initiation interval. [5]

The tutorial's top function illustrates ordered decisions. This compact form preserves that behaviour:

```c
long func(int j, int k, int c, int d) {
  int i;
  if (c > 2)      i = j - k;
  else if (d < 5) i = j + k;
  else           i = 12;
  return i;
}
```

Inputs with `j=9`, `k=4`	Selected branch	Returned value
`c=3`, `d=4`	First condition wins even though the second comparison is also true	5
`c=2`, `d=4`	First false, second true	13
`c=2`, `d=5`	Both false; constant branch	12

Preserve comparison boundaries and branch priority. Returning `long` does not automatically widen an earlier `int` operation: avoid signed overflow and check the tool's type interpretation. The handout's surrounding software driver uses uninitialized local inputs; initialize test data before treating software output as a numerical reference.

Architecture trade-off: one shared multiplier can save arithmetic resources but adds selection, storage and scheduling restrictions. More resources or pipelining may improve throughput at an area/power cost. Check actual generated reports rather than assuming a source line maps to one gate or one cycle.

Latency needs a defined input-acceptance event and matching valid-output event. **Initiation interval (II)** is spacing between accepted transactions. At 80 MHz with II=2, ideal steady-state throughput is 40 million transactions/s if interfaces do not stall; latency can be a different number of cycles. Generated RTL must also preserve transaction pairing and handshake behaviour.

Verification compares numerical results and protocol/cycle behaviour with defined inputs, including boundaries. Running the C program alone does not verify the generated RTL. Historical AppImage/install commands belong to the course demonstration; the revision does not claim a new Bambu installation or synthesis run.

10. Final recall and source trail

Before practice, explain these without looking back:

1. Why can a `reg` describe combinational logic? How does a `wire` obtain its value?

2. How do width and signedness alter a stored literal, shift or sum?

3. Distinguish bitwise, logical and reduction operators; then explain `==`, `===`, `casez` and `casex`.

4. Distinguish module type, instance, parameter, function and task.

5. Why can `initial` finish after time zero? Which unknown transitions count as edges?

6. Trace a two-register NBA pipeline and a delayed-RHS sample. Identify when each RHS is read.

7. Explain active, inactive and NBA work, settled observation and a testbench race.

8. Explain why full code coverage, a waveform and a compiled design are not equivalent to specification correctness.

9. Identify Bambu's input/output abstractions and separate latency, II and throughput.

Sources and deeper reading

1. Official NPTEL course and weekly syllabus.

2. Lesson 11: Introduction to Verilog I.

3. Lesson 12: Introduction to Verilog II.

4. Lesson 13: Functional verification using simulation.

5. Lesson 14: Bambu HLS tutorial.

6. Sutherland's Verilog-2001 reference guide: traditional language rules; SystemVerilog extends several of them.

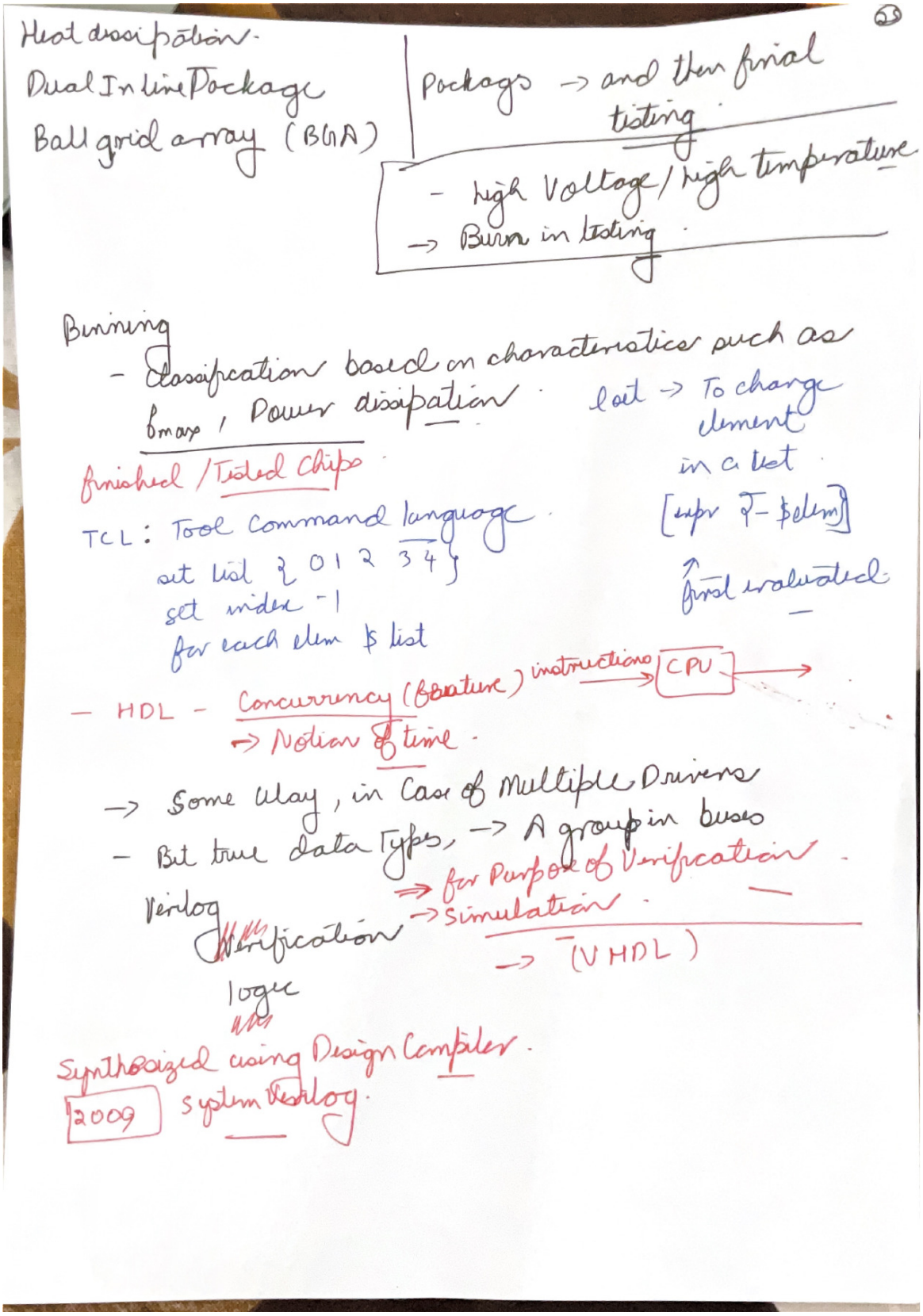
7. Official Icarus simulation documentation: compilation, elaboration and simulation workflow.

Handwriting map: the next six pages preserve Part 2 original pages 2-4 and Scan A pages 1-3. Part 2 page 2 also retains earlier Week 2 notes; its HDL section begins the Week 3 material. The typed explanations clarify shorthand about `reg`, unknowns/wildcards, functions/tasks and NBA timing. No separate handwritten Bambu page is present in the collection.

Full lecture explanations and figures remain in the Day 02 reading PDF (Lessons 11-12) and Day 03 reading PDF (Lessons 13-14). Later Day 03 synthesis/optimization lessons belong to Week 4. The handwritten index and editable language examples provide the detailed source trail.

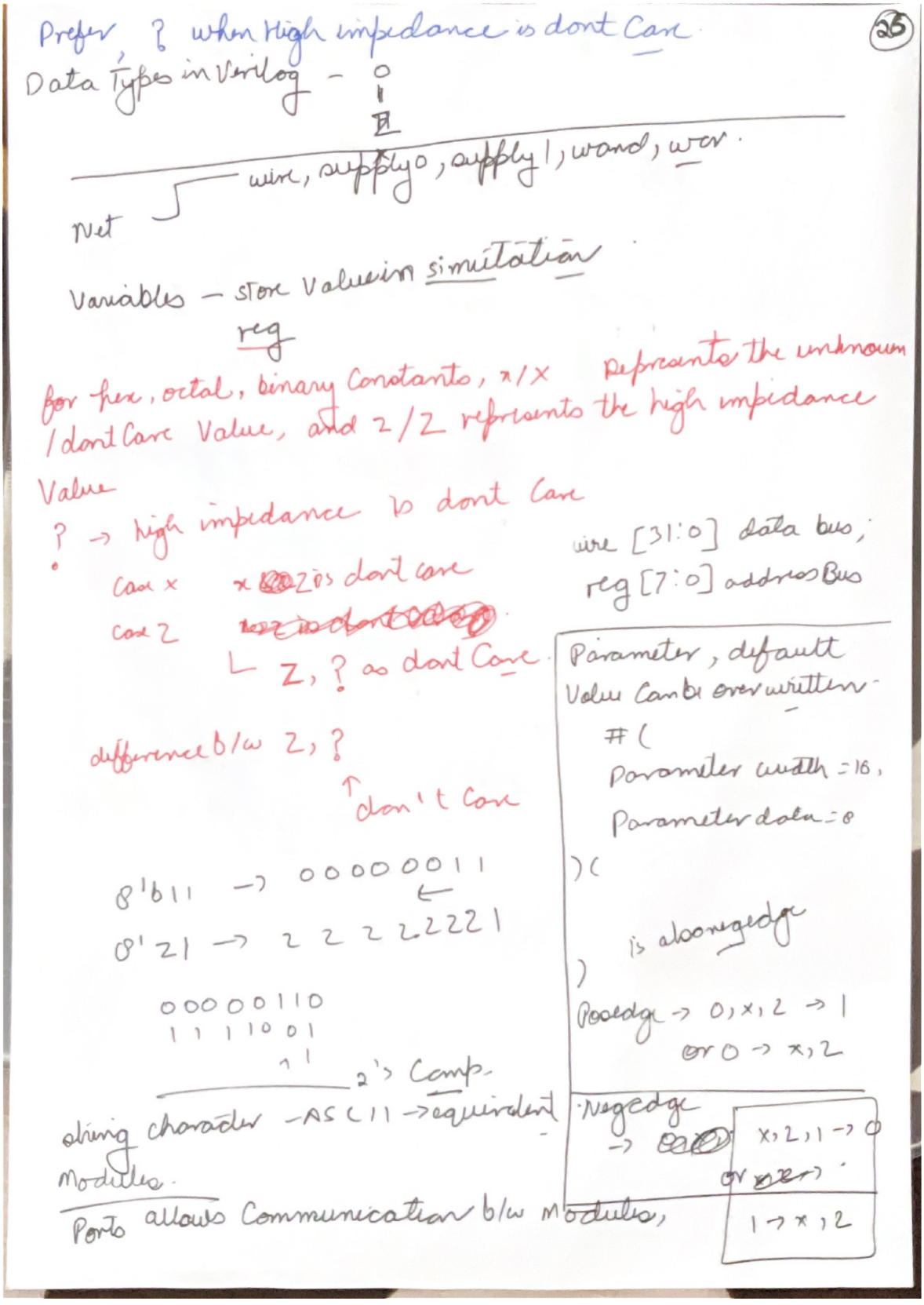
Your notes: HDL foundations

Complete Part 2 original page 2. The lower HDL section introduces Week 3 concurrency, time, multiple drivers and bit-accurate data; the earlier package/Tcl material belongs to Week 2. See typed section 1.



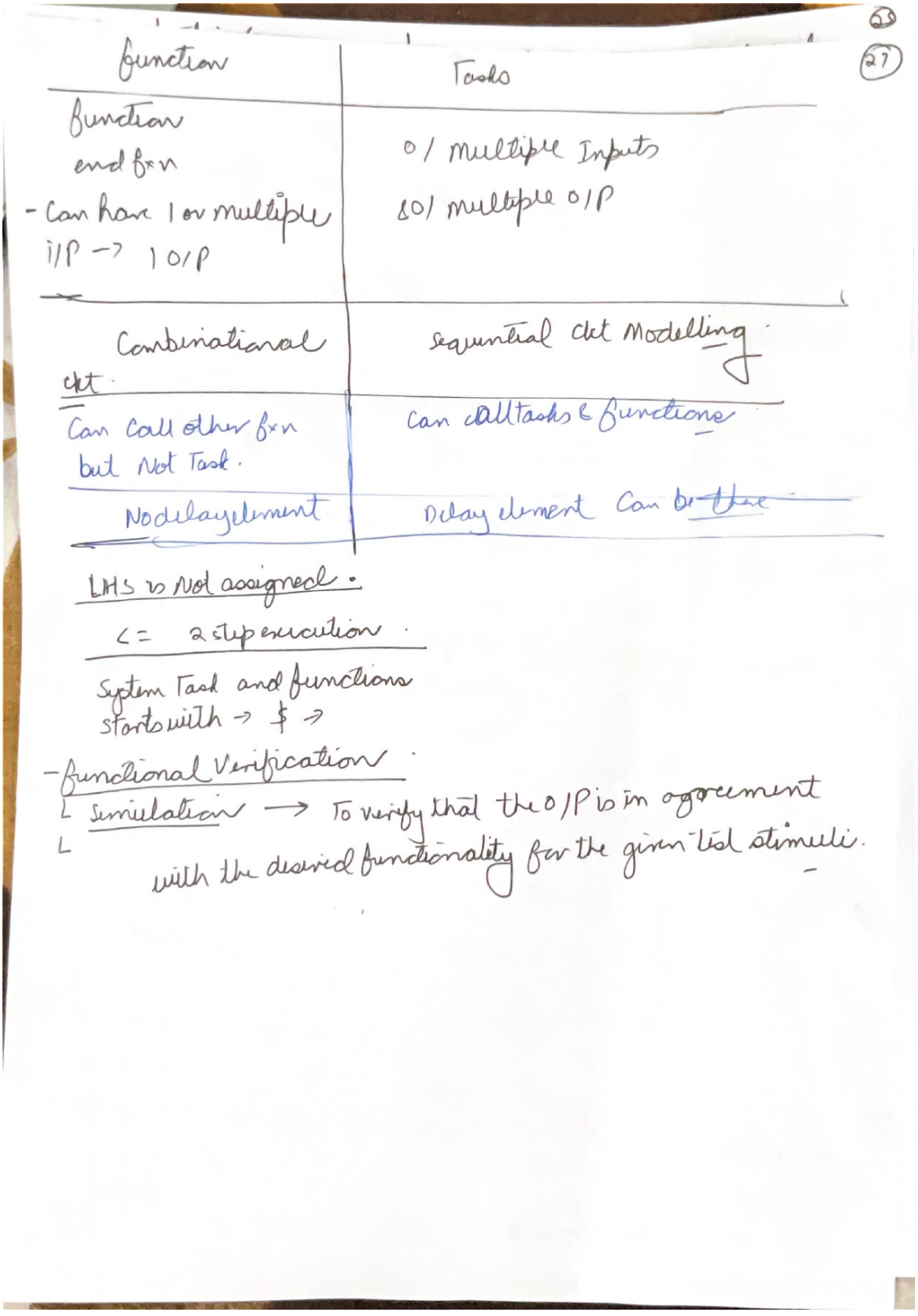
Your notes: values, types and events

Complete Part 2 original page 3. Typed sections 1-5 explain nets/variables, unknowns and wildcards, sized constants, parameters, strings, modules and four-state edge events.



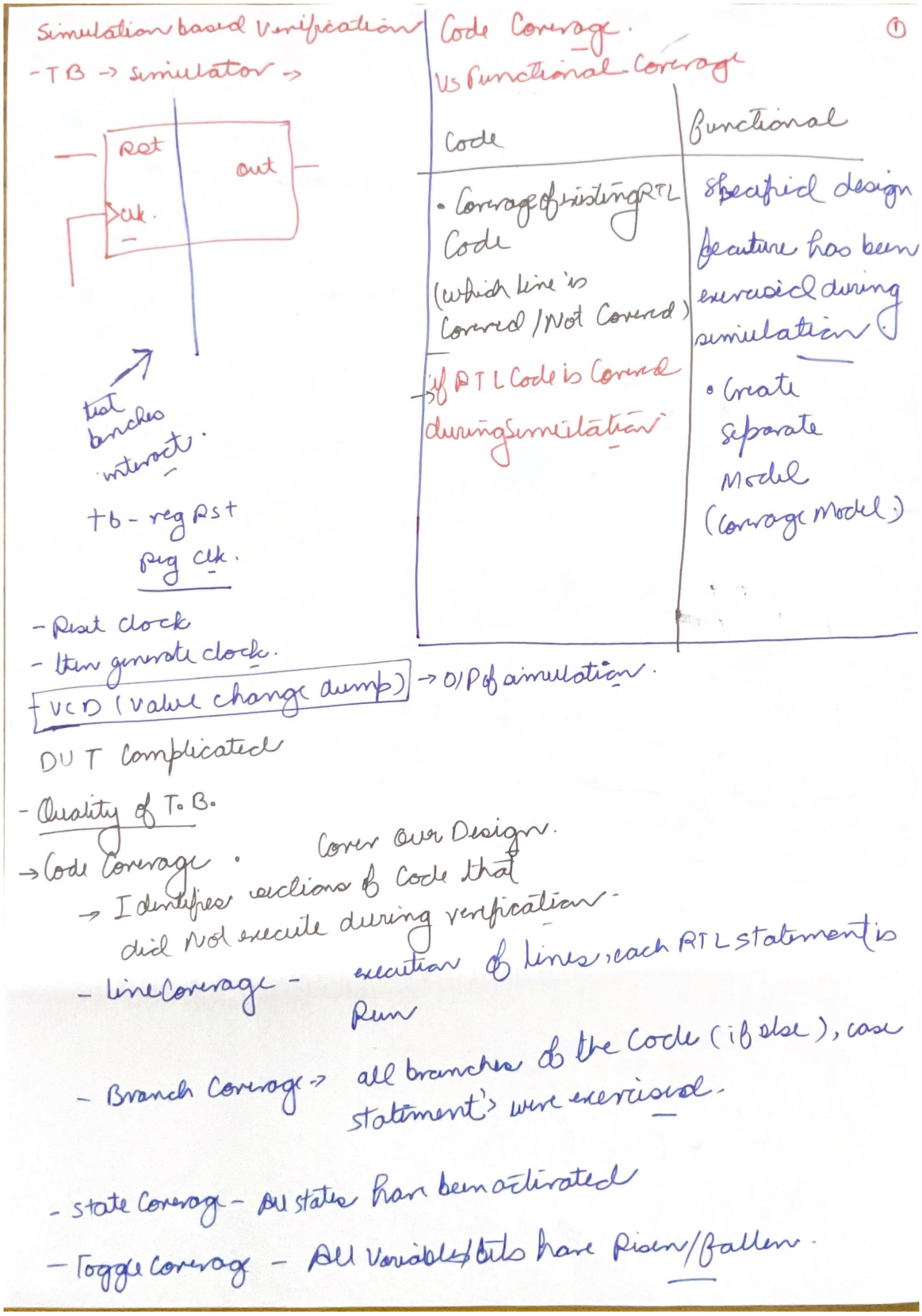
Your notes: functions and assignments

Complete Part 2 original page 4. Typed sections 4, 6 and 8 clarify traditional function/task timing, RHS evaluation versus NBA updates, system tasks and functional verification.



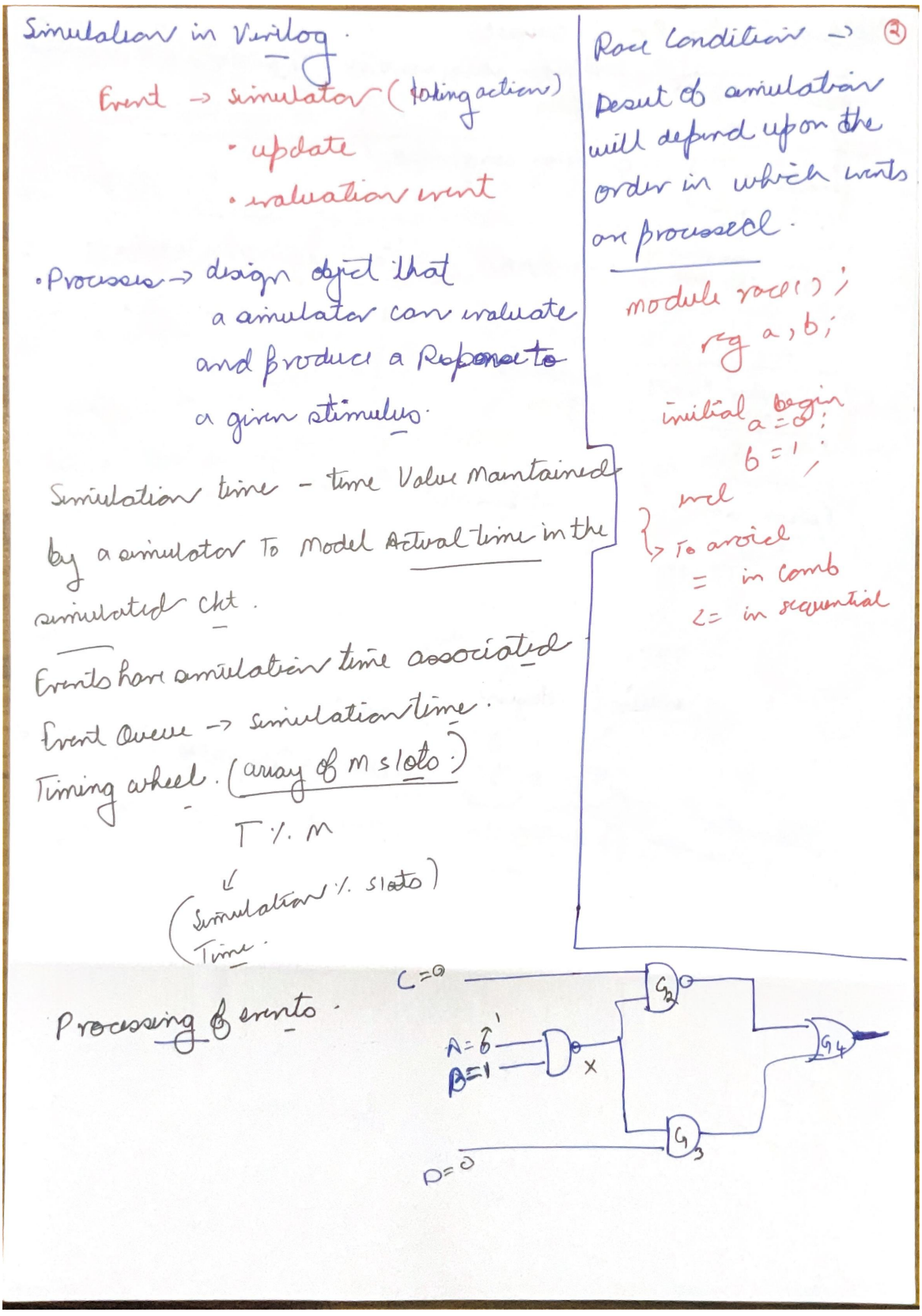
Your notes: testbench and coverage

Complete Scan A original page 1. Typed section 8 explains DUT, clock/reset stimulus, VCD, line/branch/state/toggle coverage and a separate functional coverage model.



Your notes: events and races

Complete Scan A original page 2. Typed sections 6-7 explain evaluation/update events, simulation time, the illustrative time wheel, event propagation and race conditions.



Your notes: stratified event queue

Complete Scan A original page 3. Typed section 7 traces the classic Verilog event regions and why display and settled observations can see different values at one timestamp.

